

ML course season 2

Подбор гиперпараметров и интерпретируемость моделей

22 апреля 2026 г.

Содержание

1	Гиперпараметры: что это и почему важен правильный подбор	1
1.1	Параметры vs гиперпараметры	1
1.2	Почему «просто перебрать» опасно	1
2	Кросс-валидация	2
2.1	KFold	2
2.2	StratifiedKFold	2
2.3	Nested CV (вложенная кросс-валидация)	2
3	Стратегии поиска гиперпараметров	3
3.1	Grid Search	3
3.2	Random Search	3
3.3	Bayesian Optimization: концепт	4
4	Early Stopping и настройка пайплайна	5
4.1	Early Stopping: когда хватит?	5
4.2	Пайплайн: совместный подбор препроцессинга и модели	6
4.3	Постановка эксперимента и трекинг результатов	7
5	Интерпретируемость моделей	7
5.1	Зачем нужна интерпретируемость?	7
5.2	Global vs Local объяснения	8
6	Permutation Importance (PI)	8
6.1	Идея метода	8
6.2	Плюсы, минусы, влияние корреляций	9
7	PDP и ICE-кривые	9
7.1	Partial Dependence Plot (PDP)	9
7.2	ICE-кривые	10
8	SHAP: SHapley Additive exPlanations	10
8.1	Идея: вклады признаков	10
8.2	TreeSHAP: эффективная реализация для деревьев	11
8.3	Чтение summary plot и force plot	12
8.4	SHAP vs PI vs PDP: сравнение	12
9	Практика: поиск утечек и диагностика странных признаков	13

1. Гиперпараметры: что это и почему важен правильный подбор

1.1. Параметры vs гиперпараметры

В машинном обучении принято различать два вида настроек модели.

Параметры — это те величины, которые модель *сама* узнаёт из данных в процессе обучения. Например, веса $\mathbf{w}$ логистической регрессии, средние μ и дисперсии σ^2 в Наивном Байесе, опорные векторы в SVM.

Гиперпараметры — это настройки, которые задаёт исследователь *до* запуска обучения. Они определяют «форму» задачи оптимизации, но не находятся в ходе самой оптимизации.

Примеры гиперпараметров

- Скорость обучения η (learning rate) — для градиентного спуска
- Параметр регуляризации C (или λ) — в логистической регрессии, SVM
- Параметр ядра γ — в SVM с RBF-ядром
- Максимальная глубина дерева `max_depth`, число деревьев `n_estimators`
- Число соседей k — в KNN

1.2. Почему «просто перебрать» опасно

На первый взгляд кажется, что можно просто попробовать несколько значений гиперпараметров, посмотреть, что даёт лучшую метрику на тестовой выборке — и выбрать это. Но здесь кроется принципиальная ловушка.

Утечка через тестовую выборку

Если мы подбираем гиперпараметры по результатам на тестовой выборке, то тестовая выборка **участвует в принятии решений**. Это означает, что она уже не является независимой оценкой. Мы случайно выбрали гиперпараметры, которые хорошо работают именно на этом конкретном наборе данных. Это называется **data leakage** (утечка данных). Результат: оптимистичная оценка, модель хуже работает в продакшне.

Правильное решение — использовать **валидационную выборку** или **кросс-валидацию** для подбора гиперпараметров, а тестовую выборку зарезервировать для финальной и единственной оценки.

2. Кросс-валидация

2.1. KFold

KFold — стандартная схема кросс-валидации, при которой данные делятся на k равных частей (фолдов).

KFold Cross-Validation

1. Делим обучающую выборку на k равных частей: $F_1, F_2, \dots, F_k$.
2. Для каждого $i = 1, \dots, k$:
 - Обучаем модель на $\bigcup_{j \neq i} F_j$
 - Вычисляем метрику на F_i
3. Итоговая оценка — среднее: $\bar{S} = \frac{1}{k} \sum_{i=1}^k S_i$
Обычно берут $k = 5$ или $k = 10$.

Типичное значение $k = 5$ означает: модель обучается 5 раз, каждый раз на 80% данных, и оценивается на оставшихся 20%. Итоговое значение метрики усредняется, что снижает дисперсию оценки по сравнению с однократным разбиением.

2.2. StratifiedKFold

При классификации важна пропорция классов. Если в данных 90% класса 0 и 10% класса 1, случайное разбиение может дать фолд, в котором класса 1 почти нет — тогда метрика будет нестабильной.

StratifiedKFold

StratifiedKFold гарантирует, что в каждом фолде пропорции классов соответствуют пропорциям в исходной выборке. Это особенно критично при **дисбалансе классов**.

В `sklearn` при использовании `cross_val_score` с классификатором **StratifiedKFold** применяется автоматически.

2.3. Nested CV (вложенная кросс-валидация)

Даже если мы подбираем гиперпараметры по CV (не по тесту), есть более тонкая проблема: если мы выбираем лучшие гиперпараметры по CV-оценке на всей обучающей выборке, а потом используем ту же CV-оценку как оценку качества — она **оптимистична**, потому что мы уже «искали» хорошие гиперпараметры именно для этих данных.

Nested Cross-Validation

Два уровня кросс-валидации:

- **Внешний цикл** (outer CV): оценивает качество модели. На каждой итерации отделяет тестовый фолд.
- **Внутренний цикл** (inner CV): на оставшихся данных ищет лучшие гиперпараметры.

Финальная метрика — среднее по внешним фолдам, каждый из которых «не видел» процесс подбора гиперпараметров.

На практике nested CV применяется в академических исследованиях и при строгом сравнении алгоритмов. В индустрии чаще обходятся обычным CV + зарезервированным тестом, понимая, что небольшой оптимизм приемлем.

3. Стратегии поиска гиперпараметров

3.1. Grid Search

Grid Search (поиск по сетке) — самый простой и наглядный метод. Задаём конечное множество значений для каждого гиперпараметра, а алгоритм перебирает *все* возможные комбинации.

```
1 from sklearn.model_selection import GridSearchCV
2 from sklearn.svm import SVC
3
4 param_grid = {
5     'C': [0.1, 1, 10, 100],
6     'gamma': ['scale', 'auto', 0.01, 0.001],
7     'kernel': ['rbf', 'linear'],
8 }
9
10 # 4 * 4 * 2 = 32 конфигурации, каждая проверяется 5-fold CV
11 # Итого: 32 * 5 = 160 обучений!
12 search = GridSearchCV(SVC(), param_grid, cv=5,
13                       scoring='f1_macro', n_jobs=-1)
14 search.fit(X_train, y_train)
15 print(search.best_params_)
```

Плюсы: гарантированно нашли лучшее из сетки; воспроизводимо; легко интерпретируется.

Минусы: взрывной рост числа конфигураций ($|H_1| \times |H_2| \times \dots$); нужно заранее знать разумные диапазоны; дискретизация непрерывных параметров.

3.2. Random Search

Random Search (случайный поиск) — вместо перебора сетки сэмплируем конфигурации случайно из заданных распределений.

```
1 from sklearn.model_selection import RandomizedSearchCV
2 from scipy.stats import loguniform, uniform
3
4 param_dist = {
5     'C': loguniform(0.01, 1000),          # log-равномерное [0.01,
6     ↪ 1000]
7     'gamma': loguniform(1e-4, 1.0),
8     'kernel': ['rbf', 'linear'],
9 }
10
11 search = RandomizedSearchCV(SVC(), param_dist,
12                             n_iter=50, cv=5,
13                             scoring='f1_macro',
```

```

13         random_state=42, n_jobs=-1)
14 search.fit(X_train, y_train)

```

Почему Random Search часто лучше Grid Search?

Bergstra & Bengio (2012) показали: если реальная функция качества зависит лишь от подмножества гиперпараметров, Random Search «покрывает» важные измерения плотнее, чем Grid Search с той же вычислительной стоимостью.

Например: пусть C важен, а γ — нет. Grid Search тратит половину бюджета на бесполезные значения γ . Random Search случайно попадёт в более широкий диапазон C .

3.3. Bayesian Optimization: концепт

Bayesian Optimization (байесовская оптимизация) — умный поиск, который *обучается* на предыдущих экспериментах, чтобы выбирать, какую конфигурацию попробовать следующей.

Принцип Bayesian Optimization

1. Строим **суррогатную модель** (surrogate) функции качества $f(\theta)$. Обычно это Gaussian Process или Tree-structured Parzen Estimator (TPE).
2. **Acquisition function** (функция приобретения) определяет «полезность» каждой точки: балансирует между exploration (исследованием неизвестных областей) и exploitation (уточнением вблизи известных хороших точек).
3. Выбираем $\theta^* = \text{argmax}$ acquisition function, запускаем эксперимент.
4. Обновляем суррогатную модель по новому результату $(*, f(*))$.
5. Повторяем.

Инструменты: optuna (TPE, удобный API), hyperopt (TPE), scikit-optimize (GP), ax (от Meta).

```

1  import optuna
2
3  def objective(trial):
4      C = trial.suggest_float('C', 1e-2, 1e3, log=True)
5      gamma = trial.suggest_float('gamma', 1e-4, 1.0, log=True)
6      kernel = trial.suggest_categorical('kernel', ['rbf',
7          ↪ 'linear'])
8      model = SVC(C=C, gamma=gamma, kernel=kernel)
9      return cross_val_score(model, X_train, y_train,
10                             cv=5, scoring='f1_macro').mean()
11
12 study = optuna.create_study(direction='maximize')
13 study.optimize(objective, n_trials=100)

```

```
13 print(study.best_params)
```

4. Early Stopping и настройка пайплайна

4.1. Early Stopping: когда хватит?

Чем дольше мы ищем гиперпараметры, тем больше вероятность найти конфигурацию, которая случайно хороша именно на нашем разбиении данных — это называется **переобучением к валидации** (overfitting to validation).

Overfitting к валидации при тюнинге

Это более тонкая проблема, чем обычное переобучение модели. Мы не меняем модель — мы многократно «заглядываем» на одну и ту же валидацию и выбираем результат. Чем больше попыток, тем выше шанс получить случайно хорошую метрику.

Критерии остановки поиска гиперпараметров:

- **Бюджет:** фиксированное число итераций (`n_trials`, `n_iter`) или время (`timeout`).
- **Patience:** нет улучшения метрики за последние k итераций.
- **ϵ -порог:** улучшение на очередном шаге меньше ϵ — останавливаемся.
- **Доверительный интервал:** оценка CV с разными `random_state` показывает стабильный результат.

```
1 # Optuna: pruning и early stopping
2 import optuna
3
4 study = optuna.create_study(direction='maximize')
5 study.optimize(objective,
6                 n_trials=200,
7                 timeout=600,          # не более 10 минут
8                 callbacks=[          # patience-like callback
9                     lambda s, t: s.stop()
10                     if len(s.trials) > 30
11                     and s.best_value == s.trials[-30].value
12                     else None
13                 ])
13
```

4.2. Пайплайн: совместный подбор препроцессинга и модели

Одна из самых распространённых ошибок — применить нормировку ко *всем* данным до кросс-валидации. Это утечка: параметры нормировки вычислены по данным, которые включают валидационный фолд.

Утечка через StandardScaler

```
1 # НЕПРАВИЛЬНО: scaler обучен на всём X_train, включая
  ↪ валидацию
2 scaler = StandardScaler()
3 X_scaled = scaler.fit_transform(X_train) # <- утечка!
4 cross_val_score(SVC(), X_scaled, y_train, cv=5)
```

Правильно: Pipeline

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.svm import SVC
4 from sklearn.model_selection import GridSearchCV
5
6 pipe = Pipeline([
7     ('scaler', StandardScaler()), # обучается только на
  ↪ train-фолде!
8     ('svm', SVC()),
9 ])
10
11 param_grid = {
12     'svm__C': [0.1, 1, 10, 100],
13     'svm__gamma': ['scale', 'auto', 0.01],
14     'svm__kernel': ['rbf', 'linear'],
15 }
16
17 search = GridSearchCV(pipe, param_grid, cv=5,
18                       scoring='f1_macro', n_jobs=-1)
19 search.fit(X_train, y_train)
```

Pipeline гарантирует: на каждом фолде `StandardScaler.fit` вызывается только на данных обучающего сплита, не «видя» валидацию.

4.3. Постановка эксперимента и трекинг результатов

```
1 import pandas as pd
2
3 # Результаты всех конфигураций
4 results = pd.DataFrame(search.cv_results_)[[
5     'params',
6     'mean_test_score',
7     'std_test_score',
8     'rank_test_score',
9     'mean_fit_time',
```

```
10 ]]
11 results = results.sort_values('rank_test_score')
12 print(results.head(10))
13
14 # Лучшие параметры
15 print("\nЛучшие параметры:", search.best_params_)
16 print(f"CV score: {search.best_score_:.4f} "
17       f"± {results.iloc[0]['std_test_score']:.4f}")
18
19 # Финальная оценка на тесте — ТОЛЬКО ОДИН РАЗ
20 from sklearn.metrics import classification_report
21 y_pred = search.predict(X_test)
22 print(classification_report(y_test, y_pred))
```

Правило одного теста: тестовая выборка используется ровно один раз — в самом конце, после окончательного выбора модели и гиперпараметров. Любое повторное обращение к тесту превращает его в дополнительную валидацию.

Полезные советы при трекинге:

- Всегда сохраняйте `random_state` — эксперименты должны воспроизводиться
- Смотрите на `std_test_score`: конфигурации с похожим средним, но меньшей дисперсией — надёжнее
- Логируйте время обучения (`mean_fit_time`) — иногда более простая модель предпочтительна
- Для долгих экспериментов используйте MLflow, Weights & Biases или хотя бы CSV-файл

5. Интерпретируемость моделей

5.1. Зачем нужна интерпретируемость?

Высокая точность модели — необходимое, но не достаточное условие её пригодности. Существуют три главные причины изучать, *почему* модель делает те или иные предсказания.

Зачем интерпретируемость?

1. **Доверие (Trust):** пользователи, заказчики и регуляторы хотят понимать логику решений. Кредитный скоринг, медицинская диагностика, кадровые решения — без объяснений сложно обосновать и оспорить результат.
2. **Отладка (Debugging):** интерпретация помогает найти проблемы в модели — утечку данных, артефакты выборки, нестабильные признаки. Модель с $AUC = 0.99$ может «обманывать» за счёт случайного идентификатора.

3. **Соответствие требованиям (Compliance):** GDPR в Европе даёт право на объяснение алгоритмического решения. В ряде отраслей (финансы, здравоохранение) интерпретируемость — обязательное требование.

5.2. Global vs Local объяснения

Два вопроса

Global: «Какие признаки важны для этой модели в целом?»

Отвечаем: Feature Importance, Permutation Importance, PDP.

Local: «Почему модель предсказала именно это для данного конкретного пациента/клиента?»

Отвечаем: SHAP values, LIME, force plots.

Важно понимать: глобальная важность признака X_j не означает, что он важен для *каждого* объекта. Обратное тоже верно. SHAP позволяет агрегировать локальные объяснения в глобальную картину.

6. Permutation Importance (PI)

6.1. Идея метода

Permutation Importance (PI) — модельно-независимый метод оценки важности признаков. Он отвечает на вопрос: насколько ухудшится качество модели, если мы «сломаем» связь между признаком x_j и целевой переменной?

Алгоритм Permutation Importance

1. Обучить модель, вычислить базовую метрику S_0 на валидационной выборке.
 2. Для каждого признака $j = 1, \dots, d$:
 - (a) Перемешать значения признака j случайно, сохранив все остальные
 - (b) Вычислить метрику S_j на изменённых данных
 - (c) $PI_j = S_0 - S_j$
 3. Отсортировать признаки по убыванию PI_j
- Большой PI_j = признак важен. $PI_j \approx 0$ = признак не помогает модели.

```
1 from sklearn.inspection import permutation_importance
2 import matplotlib.pyplot as plt
3
4 result = permutation_importance(
5     model, X_val, y_val,
6     n_repeats=10,      # повторяем перемешивание 10 раз
7     random_state=42,
8     scoring='f1_macro'
9 )
10
11 # Строим bar plot
```

```

12 idx = result.importances_mean.argsort()[::-1]
13 plt.barh(X_val.columns[idx], result.importances_mean[idx])
14 plt.xlabel('Снижение F1')
15 plt.title('Permutation Importance')

```

6.2. Плюсы, минусы, влияние корреляций

Плюсы:

- Работает с **любой** моделью — не нужны параметры или градиенты
- Прост в реализации и интерпретации
- Учитывает реальный вклад в метрику, а не внутренние веса модели

Минусы и ловушки:

- **Коррелированные признаки:** если x_1 и x_2 сильно коррелируют, перемешивание x_1 мало влияет на модель — она переключается на x_2 . PI обоих признаков будет занижен.
- Нужна отдельная валидационная выборка — считать PI на train некорректно.
- Результат зависит от метрики: при разных **scoring** важность может меняться.

Коллинеарность и PI

Имеем признаки «площадь квартиры» и «число комнат», которые сильно коррелируют. Перемешаем «площадь» — модель опирается на «число комнат» и почти не теряет в качестве. PI площади занижен, PI числа комнат занижен. На самом деле оба важны!

Решение: рассматривать группы коррелированных признаков вместе или использовать conditional PI.

7. PDP и ICE-кривые

7.1. Partial Dependence Plot (PDP)

PDP показывает, как изменяется *среднее предсказание модели* при изменении одного признака, при маргинализации по всем остальным.

Формула PDP

$$\hat{f}_j(x_j) = \mathbb{E}_{\mathbf{x}_{\setminus j}} [\hat{f}(x_j, \mathbf{x}_{\setminus j})] \approx \frac{1}{n} \sum_{i=1}^n \hat{f}(x_j, x_{\setminus j}^{(i)})$$

Для каждого значения x_j из сетки: подставляем его ко всем объектам выборки (заменяя их j -й признак), считаем предсказания, берём среднее.

```

1  from sklearn.inspection import PartialDependenceDisplay
2
3  # PDP для признака с индексом 0
4  PartialDependenceDisplay.from_estimator(
5      model, X_train,
6      features=[0, 1, (0, 1)], # (0,1) – двумерный PDP
7      kind='both',             # 'average' = PDP, 'individual'
8      <math>\hookrightarrow</math> = ICE
9  )

```

7.2. ICE-кривые

ICE (Individual Conditional Expectation) — расширение PDP: вместо среднего строим отдельную кривую для *каждого* объекта. Это позволяет увидеть, насколько эффект признака одинаков для разных объектов.

Когда PDP вводит в заблуждение: если для части объектов эффект признака положительный, а для другой — отрицательный, среднее (PDP) окажется близко к нулю. ICE-кривые покажут эту неоднородность.

Ограничения PDP и ICE:

- Предполагают, что признаки **независимы** при маргинализации. При сильных корреляциях создаём «нереалистичные» комбинации.
- Двумерные PDP наглядны, но при большом числе признаков их много.
- Не учитывают **взаимодействия** более высокого порядка.

8. SHAP: SHapley Additive exPlanations

8.1. Идея: вклады признаков

SHAP — метод, который объясняет предсказание модели через вклад каждого признака. Основан на значениях Шэпли из теории кооперативных игр.

SHAP-разложение

Предсказание модели для объекта x представляется как:

$$\hat{f}(x) = \underbrace{\mathbb{E}[\hat{f}]}_{\text{базовое значение}} + \sum_{j=1}^d \phi_j(x)$$

где $\phi_j(x)$ — SHAP-значение признака j для объекта x : его «вклад» в отклонение предсказания от среднего.

Идея значений Шэпли: представим признаки как «игроков», которые вместе образуют «коалиции». Насколько ухудшится предсказание, если убрать признак j

из коалиции? Усредняем по всем возможным коалициям — получаем справедливое распределение «заслуги».

Три ключевых свойства SHAP:

- **Локальная точность:** $\mathbb{E}[\hat{f}] + \sum_j \phi_j = \hat{f}(x)$ — сумма вкладов даёт предсказание
- **Отсутствие фиктивных вкладов:** если признак не влияет на предсказание, его SHAP = 0
- **Согласованность:** если признак стал важнее, его SHAP не уменьшится

8.2. TreeSHAP: эффективная реализация для деревьев

Для ансамблей деревьев (Random Forest, GBM) существует эффективная реализация TreeSHAP с полиномиальной сложностью вместо экспоненциальной.

```

1  import shap
2
3  # Для деревьев (Random Forest, XGBoost, LightGBM и др.)
4  explainer = shap.TreeExplainer(model)
5  shap_values = explainer.shap_values(X_test)
6  # shap_values.shape: (n_samples, n_features) для бинарной
   ↪ классификации
7  # или (n_classes, n_samples, n_features) для мультикласса
8
9  # Для других моделей (медленнее, но универсально)
10 explainer_kernel = shap.KernelExplainer(model.predict_proba,
   ↪ X_train)
11 shap_values_kernel = explainer_kernel.shap_values(X_test[:50])

```

8.3. Чтение summary plot и force plot

Summary Plot: глобальная картина

```

1  # Summary plot: все объекты, все признаки
2  shap.summary_plot(shap_values, X_test,
   ↪ feature_names=X_test.columns)

```

На графике:

- **Ось Y:** признаки, отсортированные по убыванию средней важности $|\phi_j|$
- **Ось X:** SHAP-значение. Положительное = увеличивает вероятность класса 1. Отрицательное = уменьшает.
- **Цвет точки:** значение признака. Красный = высокое, синий = низкое.
- **Горизонтальное расположение точек:** множество объектов при данном значении SHAP.

Пример: если для признака «доход» красные точки (высокий доход) находятся правее нуля, это означает: высокий доход увеличивает предсказываемую

вероятность.

Force Plot: один объект

```

1 # Force plot для одного объекта
2 shap.force_plot(
3     explainer.expected_value,
4     shap_values[0],
5     X_test.iloc[0],
6     feature_names=X_test.columns
7 )

```

На force plot видно: какие признаки «толкают» предсказание вправо (повышают класс 1) и влево (понижают). Основание — базовое значение $\mathbb{E}[\hat{f}]$, итог — $\hat{f}(x)$.

8.4. SHAP vs PI vs PDP: сравнение

	PI	PDP/ICE	SHAP
Тип объяснения	Global	Global	Local + Global
Работает с любой моделью	Да	Да	Да (KernelSHAP)
Быстро для деревьев	Да	Да	Да (TreeSHAP)
Корреляции	Проблема	Проблема	Лучше справляется
Объяснение одного примера	Нет	Нет	Да
Математические гарантии	Нет	Нет	Да (Shapley)
Сложность интерпретации	Низкая	Средняя	Средняя

9. Практика: поиск утечек и диагностика странных признаков

Интерпретация — мощный инструмент отладки моделей. Рассмотрим паттерны, на которые стоит обращать внимание.

Признаки с подозрительно высокой важностью:

- Идентификаторы, индексы, временные метки — часто случайно коррелируют с целевой переменной в историческом датасете
- Признаки, которые «знают будущее» (например, дата исполнения заказа в задаче предсказания отмены)
- Агрегаты по целевой переменной — статистика по таргету, вычисленная без должной изоляции

Что делать, если нашли подозрительный признак:

1. Проверить силу корреляции с таргетом и природу признака
2. Удалить признак, переобучить, сравнить метрики

3. Изучить его SHAP: как он влияет на разные объекты?

```
1  # Сравнение PI и SHAP importances на одной модели
2  import pandas as pd
3
4  # PI
5  pi = permutation_importance(model, X_val, y_val,
6                             n_repeats=10, random_state=42)
7  pi_df = pd.DataFrame({'feature': X_val.columns,
8                       'PI':
9                       ↪ pi.importances_mean}).sort_values('PI',
10                       ↪ ascending=False)
11
12  # SHAP
13  explainer = shap.TreeExplainer(model)
14  sv = explainer.shap_values(X_val)
15  shap_df = pd.DataFrame({'feature': X_val.columns,
16                       'SHAP_mean_abs':
17                       ↪ np.abs(sv).mean(axis=0)})
18                       .sort_values('SHAP_mean_abs',
19                       ↪ ascending=False)
20
21  # Объединяем и смотрим на расхождения
22  combined = pi_df.merge(shap_df, on='feature')
23  print(combined)
```

Если порядок признаков по PI и SHAP сильно расходится — это сигнал: либо есть корреляции (PI их занижает), либо признак влияет локально (SHAP это видит, PI — нет).

Итоги и домашнее задание

Часть 1: Гиперпараметры

1. KFold/StratifiedKFold — честная оценка без утечки данных
2. Grid Search — полный перебор, Random Search — случайный сэмплинг
3. Bayesian Optimization — умный поиск через суррогатную модель
4. Pipeline — защита от leakage при совместном подборе
5. Early stopping — против переобучения к валидации

Часть 2: Интерпретируемость

1. Permutation Importance — модельно-независимая глобальная важность
2. PDP/ICE — визуализация эффекта признака
3. SHAP — локальные и глобальные объяснения с гарантиями
4. Диагностика: поиск утечек и странных признаков через интерпретацию

Домашнее задание

1. Прочитать главы о model selection и interpretability в книге Christoph Molnar «Interpretable Machine Learning»: <https://christophm.github.io/interpretable-ml-book>
2. **Выполнить семинарский ноутбук:** подбор гиперпараметров SVM/RandomForest на одном из изученных датасетов + интерпретация модели через PI, PDP, SHAP
3. Сравнить: Random Search vs Grid Search по качеству и скорости при одинаковом числе оценок

scikit-learn.org • shap.readthedocs.io • optuna.org
christophm.github.io/interpretable-ml-book